

ExamArena

Final Project Decision & Product Blueprint
Real-Time Collaborative Exam & Adaptive Learning Platform

1. Final Decision

We are building ExamArena. It is a shared, real-time virtual exam room for students. Students join a course/study room, chat normally before the exam, take a synchronized exam, receive individual performance analysis, compare their performance with the class, identify weak topics, and receive targeted practice generated from their actual study material.

The product is **not** positioned as another generic AI chatbot or PDF question generator. The core product loop is: **community** → **live exam** → **class results** → **personal weakness analysis** → **targeted practice** → **improvement**.

2. Why We Chose This Direction

We intentionally stopped searching for a completely untouched problem. Existing products already cover many obvious student and job-related ideas. The project therefore focuses on demonstrating strong software engineering while solving a meaningful student workflow.

Discarded directions: generic student collaboration/study apps, PDF RAG study assistants, job aggregation/matching, and several other operational workflow ideas that already have established products.

3. Core User Experience

1	Create / Join Room	A creator creates a course/study room. Students join using the room or course.
2	Normal Chat	Before the exam, everyone can communicate normally using real-time chat.
3	Upload Material	The creator uploads syllabus, notes, PDFs, previous questions and other course material.
4	Build Exam Blueprint	The system uses the university's official information to determine the relevant syllabus/exam structure.
5	Generate Exam	AI generates MCQ, SAQ, 5-mark and other configured question types using the blueprint and uploaded material.
6	Start Live Exam	The creator presses Start. The room changes from chat mode to synchronized exam mode.
7	Submit	Students answer under a shared timer and submit their responses.
8	Analyze	The system calculates scores and analyzes individual and class-level performance.
9	Personal Report	Each student receives strengths, weaknesses, class comparison and topic-level performance.
10	Targeted Practice	AI generates focused practice/rematch questions based on weak areas and grounded in the available study material.

4. The Product Loop

CHAT → EXAM → RESULT → CLASS COMPARISON → WEAKNESS ANALYSIS → AI REMATCH → IMPROVEMENT → NEXT EXAM

The important idea is that AI is not the reason students have to return. The shared room, classmates, live exams, performance comparison and repeated practice create the recurring reason to come back.

5. The University + Study Material Model

The system should treat two sources separately:

A. University source: used to understand syllabus, official exam structure/pattern and other authoritative information.

B. Uploaded course material: used as the knowledge base for question generation, explanations and targeted practice.

The university lookup should not happen blindly every time an exam is generated. Instead, the system should extract and store a structured exam blueprint with its source information, which can then be reused.

6. Real-Time Exam

Socket.IO is justified because the exam requires synchronized state across many students.

Example events: `room.created`, `student.joined`, `exam.started`, `question.changed`, `answer.submitted`, `timer.updated`, `exam.completed`, `leaderboard.updated`

The server should remain authoritative for exam timing, exam state, submission and final scoring rather than trusting the browser.

7. Individual Performance Report

A result should contain more than a single score. Example:

Score: 36/50
Class average: 31/50
Accuracy: 72%
Strong topics: Regression, Classification
Weak topics: Decision Trees, Ensemble Learning, ANN
Question-type performance: MCQ / SAQ / 5-mark / longer answers

The comparison should help the student understand their performance without turning the application into a simple ranking board.

8. AI + RAG

RAG is used for grounded learning, not as a generic 'ask anything' chatbot.

Course PDFs/notes/PYQs → text extraction → chunking → embeddings → vector store → retrieval → grounded explanation/question generation.

When a student is weak in a topic, the system retrieves the relevant course material and generates targeted explanations and practice.

9. Signature Feature: Rematch

After an exam, the student can enter a targeted rematch focused on weak areas.

Example: A student scores 2/6 on Deadlocks. The system creates a short Deadlocks-focused practice set from the course material. The student can then compare the new result with the original performance.

This turns the application into a learning cycle rather than a one-time quiz.

10. Proposed Technology Stack

Layer	Technology
Frontend	React + TypeScript + Tailwind + Zustand + React Router
Backend	Hono + TypeScript
Database	MongoDB initially

Real-time	Socket.IO
AI	Gemini + LangChain
RAG	Google embeddings + Pinecone
Deployment	Docker + suitable cloud services

11. Development Principles

- **Build in stages.** Do not start with AI or WebSockets before the basic exam engine works.
- **Every technology needs a reason.** Socket.IO is for synchronized real-time state; RAG is for grounded course knowledge; Docker is for reproducible deployment.
- **The student remains the developer.** AI can assist with implementation, debugging and explanation, but the important code and architecture must remain understandable.
- **Do not overbuild the MVP.** Get authentication, rooms, exams, submissions and results working first.
- **Keep the live room central.** Chat → exam → analysis → practice is the product's identity.

12. MVP Boundary

The first working version should contain only:

- Authentication
- Create/join study room
- Real-time pre-exam chat
- Upload course material
- Create/configure an exam
- MCQ exam engine
- Synchronized timer and exam state
- Answer submission and scoring
- Basic individual result report

After this is stable, add university-pattern extraction, additional question types, RAG, AI analysis, class analytics and rematch.

Project North Star: Build a virtual exam room where students don't just ask an AI questions individually. They study together, take exams together, learn from collective performance, and use AI to close their individual knowledge gaps.